PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT

DR. RVR NRI INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

Course: Python Programming
Student Name: M. Jashwanth Shalom
Roll Number: 26000039
Branch & Section: CSE (core)-1
Academic Year: 2026–2027

Case Study Index

No.	Case Study Title	Concept
1	Student Pass/Fail Checker	if-else
2	Exam Eligibility Checker	if-else
3	Scholarship Eligibility Calculator	if-elif-else, logical operator and
4	Electricity Bill Calculator	if-elif-else
5	Hospital Bill Discount Calculator	if-else, arithmetic
6	Employee Salary Allowance Calculator	if-else
7	Mini ATM System Simulation	if-elif-else, nested if
8	Student Marks Total and Average	for loop
9	Weekly Power Consumption Tracker	while loop
10	Interactive Number Guessing Game	while loop

Student Pass/Fail Checker

Python file: a1.py **Concept:** if-else

Problem Statement

Check whether a student passes based on the marks entered.

Objective

Display Pass if marks are 35 or above; otherwise display Fail.

Algorithm

1. Input marks.

2. If marks >= 35, display Pass; otherwise, display Fail.

Pseudocode

```
START
INPUT marks
IF marks >= 35
    DISPLAY "Pass"
ELSE
    DISPLAY "Fail"
END IF
STOP
```

Python Source Code

```
marks = int(input("Enter marks: "))

if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

Sample Input

```
Enter marks: 75
```

Expected Output — Text Only

```
Pass
```

Test Cases

Input / Test	Expected Result
75	Pass
28	Fail

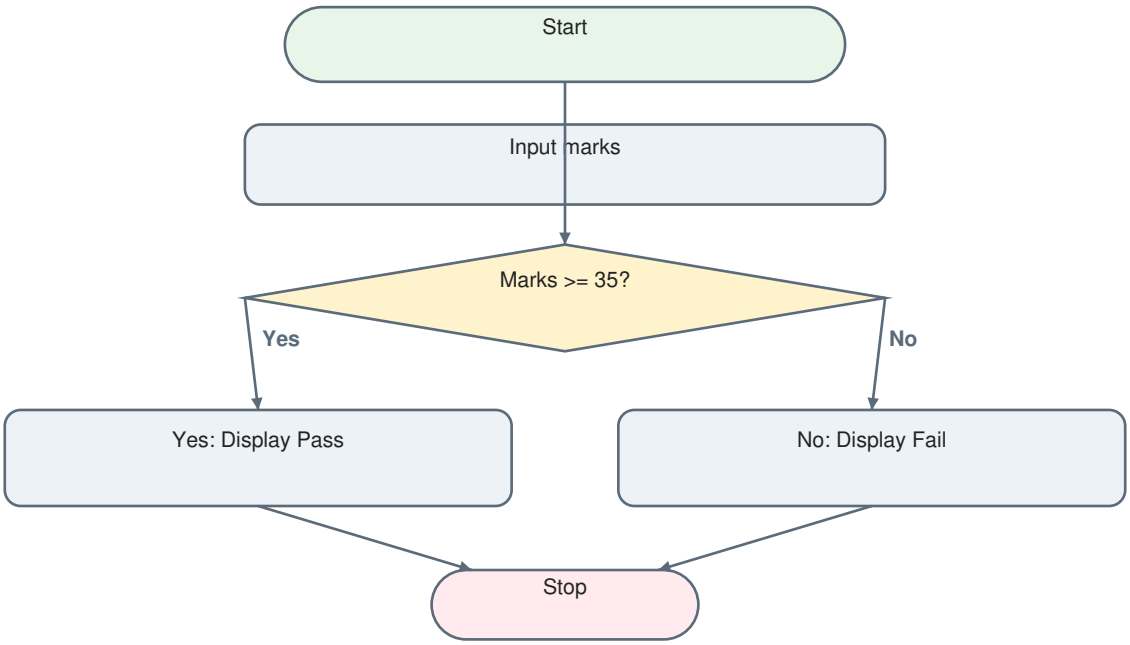
Conclusion

This case demonstrates the stated Python concept.

Experiment 1: Student Pass/Fail Checker

FLOWCHART AND OUTPUT SCREENSHOT

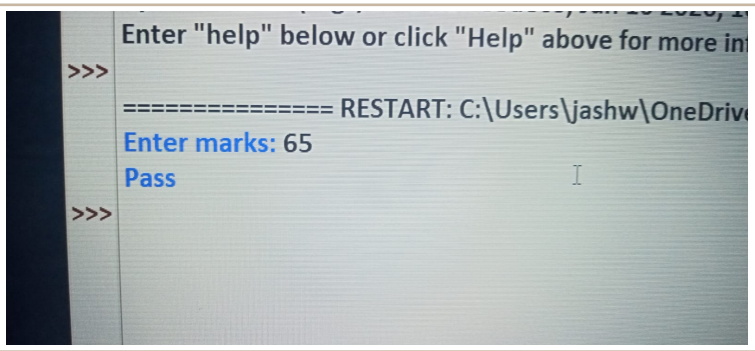
FLOWCHART



ASSUMPTIONS

Marks are entered as a whole number from 0 to 100. A score of 35 or more is a pass.

PYTHON IDLE OUTPUT SCREENSHOT



Exam Eligibility Checker

Python file: a2.py **Concept:** if-else

Problem Statement

Check exam eligibility using the attendance percentage.

Objective

Display Eligible for exams when attendance is 75% or above.

Algorithm

1. Input attendance percentage.

2. If attendance >= 75, display Eligible for exams; otherwise, display Not eligible for exams.

Pseudocode

```
START
INPUT attendance
IF attendance >= 75
    DISPLAY "Eligible for exams"
ELSE
    DISPLAY "Not eligible for exams"
END IF
STOP
```

Python Source Code

```
attendance = int(input("Enter attendance percentage: "))

if attendance >= 75:
    print("Eligible for exams")
else:
    print("Not eligible for exams")
```

Sample Input

```
Enter attendance percentage: 80
```

Expected Output — Text Only

```
Eligible for exams
```

Test Cases

Input / Test	Expected Result
80	Eligible for exams
65	Not eligible for exams

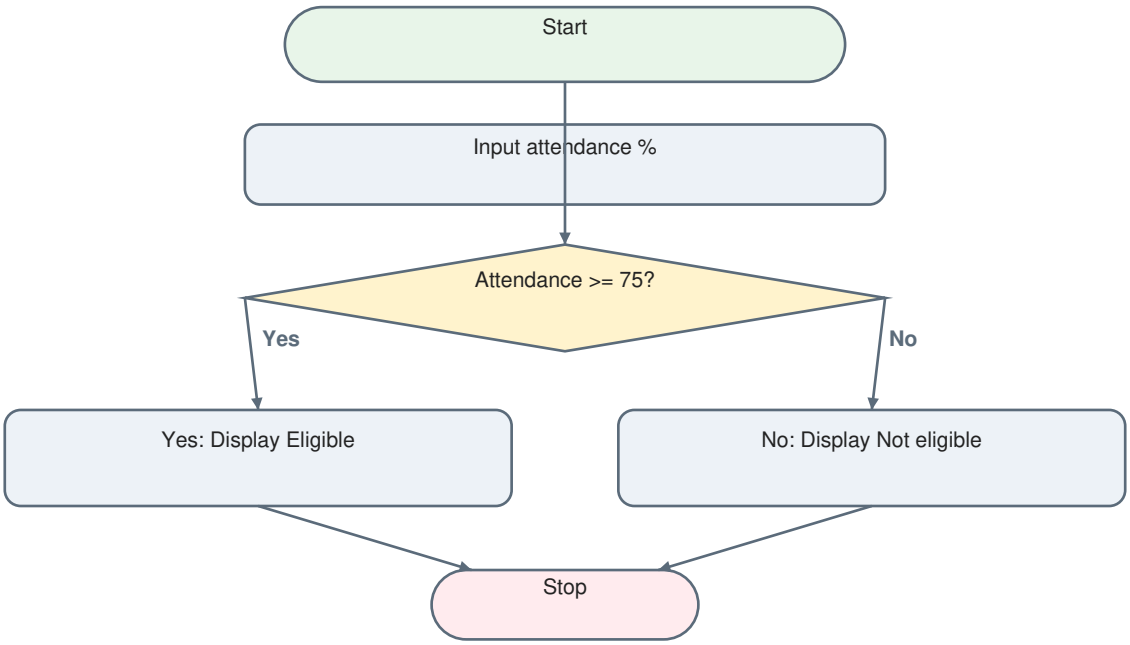
Conclusion

This case demonstrates the stated Python concept.

Experiment 2: Exam Eligibility Checker

FLOWCHART AND OUTPUT SCREENSHOT

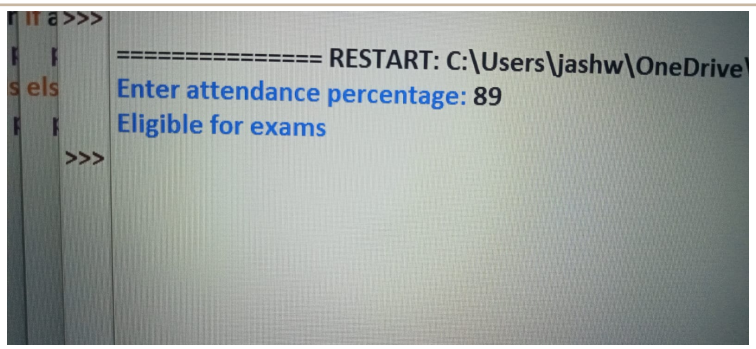
FLOWCHART



ASSUMPTIONS

Attendance is entered as a percentage from 0 to 100. The eligibility threshold is 75%.

PYTHON IDLE OUTPUT SCREENSHOT



Scholarship Eligibility Calculator

Python file: a3.py **Concept:** if-elif-else, logical operator and

Problem Statement

Determine scholarship eligibility using marks and family income.

Objective

Apply the conditions in the program to display full, half, or no scholarship.

Algorithm

1. Input marks and family income.

2. If marks >= 90 and income <= 200000, display Full scholarship.

3. Otherwise, if marks >= 75 and income <= 300000, display Half scholarship.

4. Otherwise, display No scholarship.

Pseudocode

```
START
INPUT marks, income
IF marks >= 90 AND income <= 200000
    DISPLAY "Full scholarship"
ELSE IF marks >= 75 AND income <= 300000
    DISPLAY "Half scholarship"
ELSE
    DISPLAY "No scholarship"
END IF
STOP
```

Python Source Code

```
marks = int(input("Enter marks: "))
income = int(input("Enter family income: "))

if marks >= 90 and income <= 200000:
    print("Full scholarship")
elif marks >= 75 and income <= 300000:
    print("Half scholarship")
else:
    print("No scholarship")
```

Sample Input

```
Enter marks: 95
Enter family income: 150000
```

Expected Output — Text Only

```
Full scholarship
```

Test Cases

Input / Test	Expected Result
95, 150000	Full scholarship
80, 250000	Half scholarship
70, 400000	No scholarship

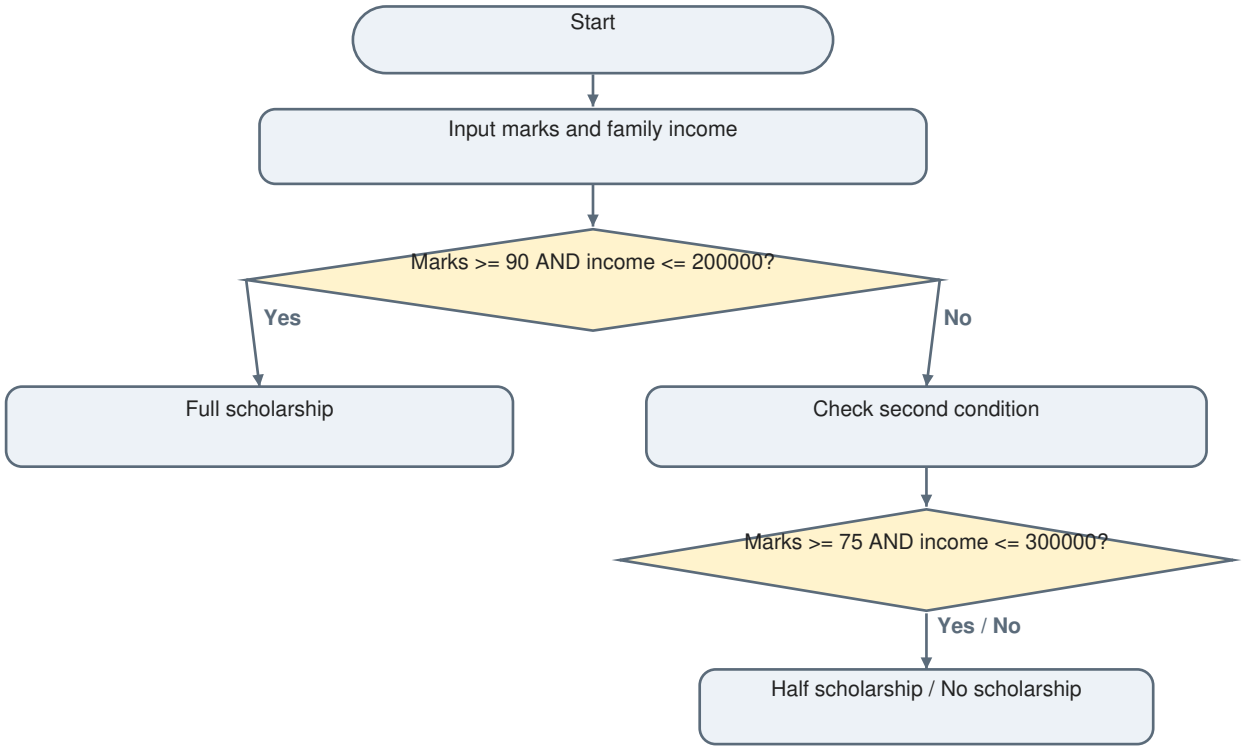
Conclusion

This case demonstrates the stated Python concept.

Experiment 3: Scholarship Eligibility Calculator

FLOWCHART AND OUTPUT SCREENSHOT

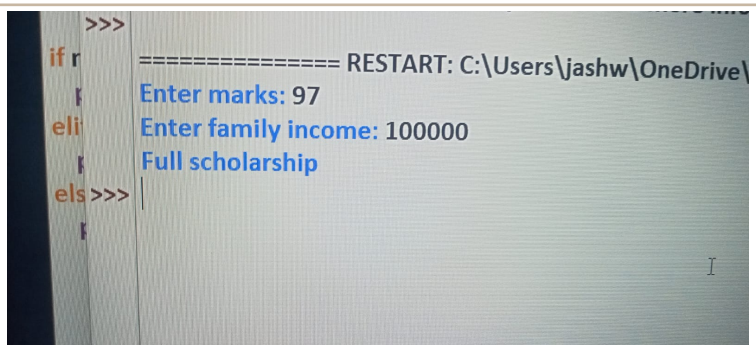
FLOWCHART



ASSUMPTIONS

Marks and family income are valid non-negative numbers. The higher scholarship condition is checked first.

PYTHON IDLE OUTPUT SCREENSHOT



Electricity Bill Calculator

Python file: a4.py **Concept:** if-elif-else

Problem Statement

Calculate an electricity bill based on units consumed.

Objective

Use the rates specified in the supplied code: up to 100 units at 2 per unit, up to 200 units at 3 per unit, and above 200 units at 5 per unit.

Algorithm

1. Input units consumed.
2. If units <= 100, bill = units * 2.
3. Otherwise, if units <= 200, bill = units * 3.
4. Otherwise, bill = units * 5.
5. Display the bill.

Pseudocode

```
START
INPUT units
IF units <= 100
    bill = units * 2
ELSE IF units <= 200
    bill = units * 3
ELSE
    bill = units * 5
END IF
DISPLAY bill
STOP
```

Python Source Code

```
units = int(input("Enter units consumed: "))

if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = units * 3
else:
    bill = units * 5

print("Electricity bill =", bill)
```

Sample Input

```
Enter units consumed: 80
```

Expected Output — Text Only

```
Electricity bill = 160
```

Test Cases

Input / Test	Expected Result
80	Electricity bill = 160
150	Electricity bill = 450
250	Electricity bill = 1250

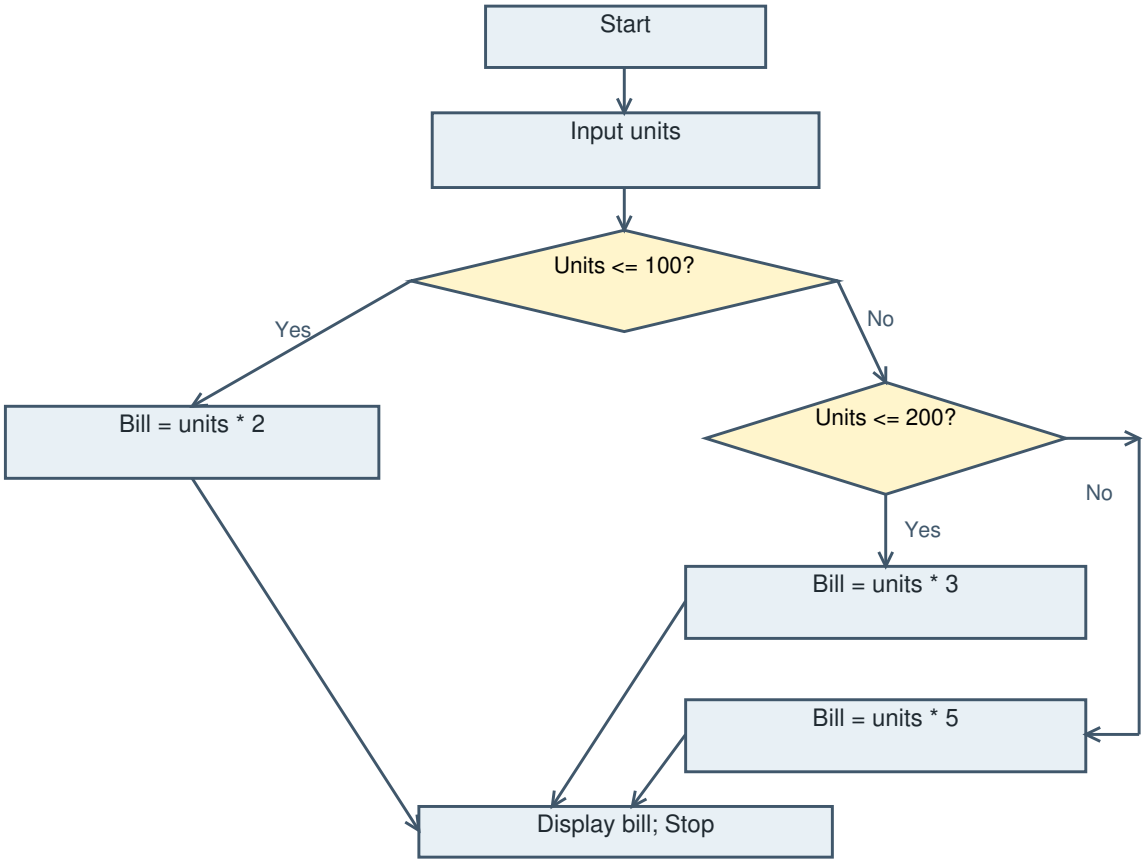
Conclusion

This case demonstrates the stated Python concept.

Experiment 4: Electricity Bill Calculator

FLOWCHART AND OUTPUT SCREENSHOT

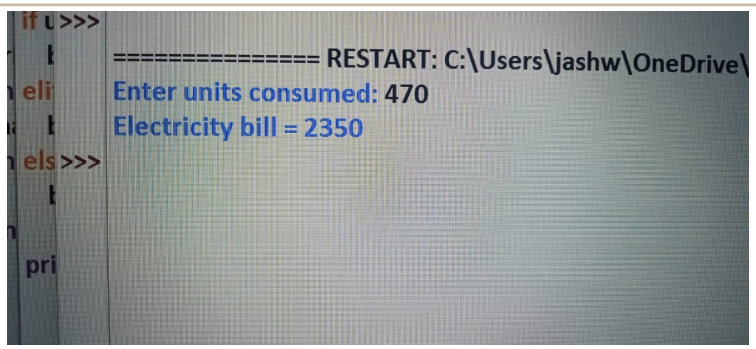
FLOWCHART



ASSUMPTIONS

Units consumed are non-negative whole numbers. The program applies one rate to all units based on the selected range.

PYTHON IDLE OUTPUT SCREENSHOT



Hospital Bill Discount Calculator

Python file: a5.py **Concept:** if-else, arithmetic

Problem Statement

Calculate a discount and final amount for a hospital bill.

Objective

Apply a 10% discount when the bill is ■5,000 or more; otherwise apply no discount.

Algorithm

1. Input hospital bill.

2. If bill >= 5000, calculate a 10% discount; otherwise set discount to 0.

3. Subtract the discount from the bill.

4. Display discount and final bill.

Pseudocode

```
START
INPUT bill
IF bill >= 5000
    discount = bill * 10 / 100
ELSE
    discount = 0
END IF
total = bill - discount
DISPLAY discount, total
STOP
```

Python Source Code

```
bill = int(input("Enter hospital bill: "))

if bill >= 5000:
    discount = bill * 10 / 100
else:
    discount = 0

total = bill - discount

print("Discount =", discount)
print("Final bill =", total)
```

Sample Input

```
Enter hospital bill: 6000
```

Expected Output — Text Only

```
Discount = 600.0
Final bill = 5400.0
```

Test Cases

Input / Test	Expected Result
6000	Discount = 600.0; Final bill = 5400.0
4000	Discount = 0; Final bill = 4000

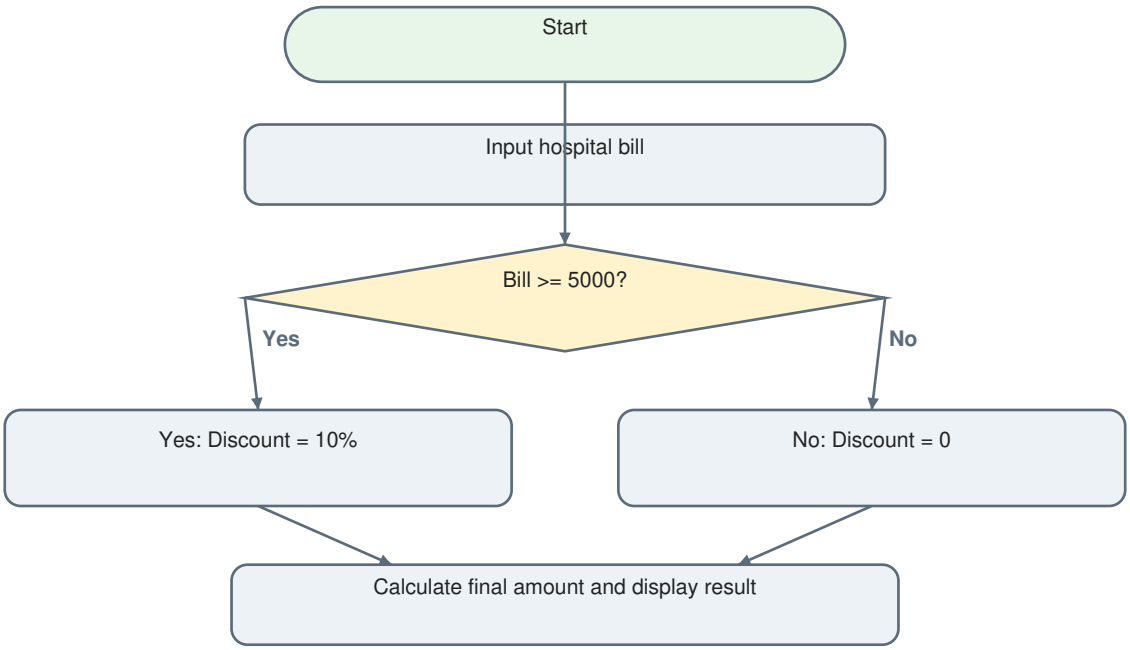
Conclusion

This case demonstrates the stated Python concept.

Experiment 5: Hospital Bill Discount Calculator

FLOWCHART AND OUTPUT SCREENSHOT

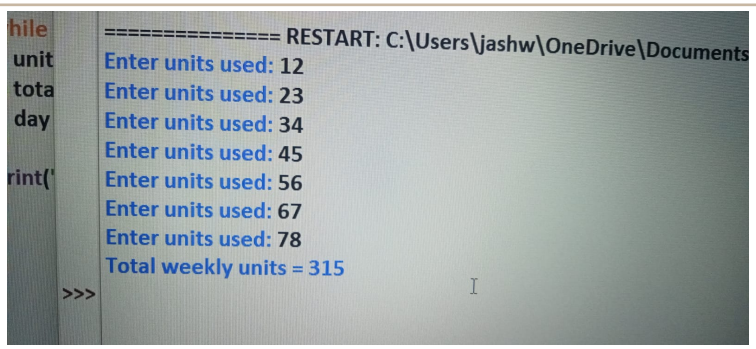
FLOWCHART



ASSUMPTIONS

The hospital bill is non-negative. Bills of 5,000 or more receive a 10% discount.

PYTHON IDLE OUTPUT SCREENSHOT



Employee Salary Allowance Calculator

Python file: a6.py **Concept:** if-else

Problem Statement

Calculate an employee's allowance and total salary from the basic salary.

Objective

Give an allowance of 5000 when salary is at least 20000; otherwise give 2000.

Algorithm

1. Input basic salary.

2. If salary >= 20000, set allowance to 5000; otherwise set it to 2000.

3. Add allowance to salary.

4. Display allowance and total salary.

Pseudocode

```
START
INPUT salary
IF salary >= 20000
    allowance = 5000
ELSE
    allowance = 2000
END IF
total = salary + allowance
DISPLAY allowance, total
STOP
```

Python Source Code

```
salary = int(input("Enter basic salary: "))

if salary >= 20000:
    allowance = 5000
else:
    allowance = 2000

total = salary + allowance

print("Allowance =", allowance)
print("Total salary =", total)
```

Sample Input

```
Enter basic salary: 25000
```

Expected Output — Text Only

```
Allowance = 5000
Total salary = 30000
```

Test Cases

Input / Test	Expected Result
25000	Allowance = 5000; Total salary = 30000
15000	Allowance = 2000; Total salary = 17000

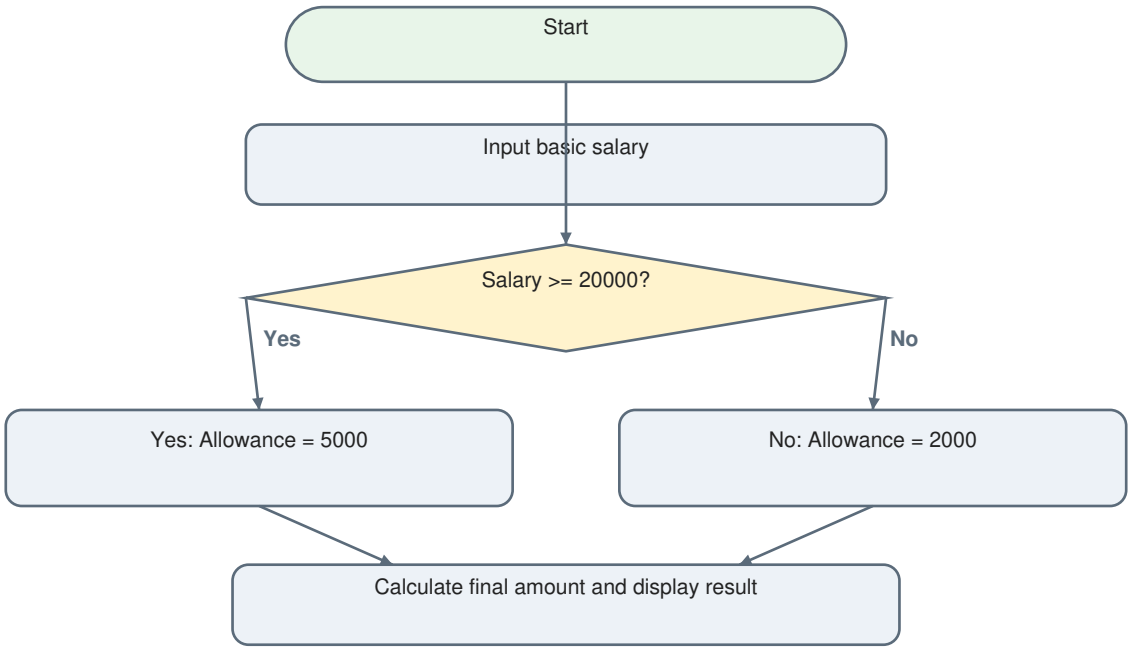
Conclusion

This case demonstrates the stated Python concept.

Experiment 6: Employee Salary Allowance Calculator

FLOWCHART AND OUTPUT SCREENSHOT

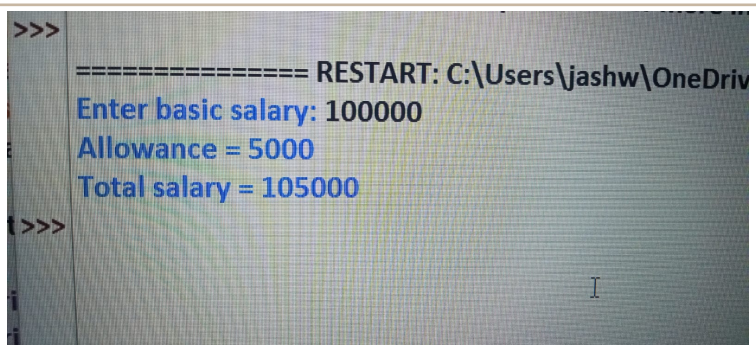
FLOWCHART



ASSUMPTIONS

Basic salary is a non-negative amount. Salary of 20,000 or more receives 5,000 allowance; otherwise 2,000.

PYTHON IDLE OUTPUT SCREENSHOT



Mini ATM System Simulation

Python file: a7.py **Concept:** if-elif-else, nested if

Problem Statement

Simulate checking balance, depositing money, and withdrawing money.

Objective

Use menu selection to update a starting balance of 5000 and reject withdrawals greater than the balance.

Algorithm

1. Set balance = 5000 and display the menu.

2. Input choice.

3. Choice 1 displays the balance; choice 2 deposits an amount; choice 3 withdraws only if sufficient funds exist.

4. For any other choice, display Invalid choice.

Pseudocode

```
START
balance = 5000
DISPLAY menu
INPUT choice
IF choice == 1
    DISPLAY balance
ELSE IF choice == 2
    INPUT amount
    balance = balance + amount
    DISPLAY balance
ELSE IF choice == 3
    INPUT amount
    IF amount <= balance
        balance = balance - amount
        DISPLAY balance
    ELSE
        DISPLAY "Insufficient balance"
    END IF
ELSE
    DISPLAY "Invalid choice"
END IF
STOP
```

Python Source Code

```
balance = 5000

print("1. Check Balance")
print("2. Deposit")
print("3. Withdraw")

choice = int(input("Enter choice: "))

if choice == 1:
    print("Balance =", balance)
elif choice == 2:
    amount = int(input("Enter deposit amount: "))
    balance = balance + amount
    print("Balance =", balance)
elif choice == 3:
    amount = int(input("Enter withdrawal amount: "))
    if amount <= balance:
        balance = balance - amount
        print("Balance =", balance)
    else:
        print("Insufficient balance")
else:
    print("Invalid choice")
```

Sample Input

```
Enter choice: 2
Enter deposit amount: 1000
```

Expected Output — Text Only

```
1. Check Balance
```

```
2. Deposit
3. Withdraw
Balance = 6000
```

Test Cases

Input / Test	Expected Result
1	Balance = 5000
2, then 1000	Balance = 6000
3, then 6000	Insufficient balance

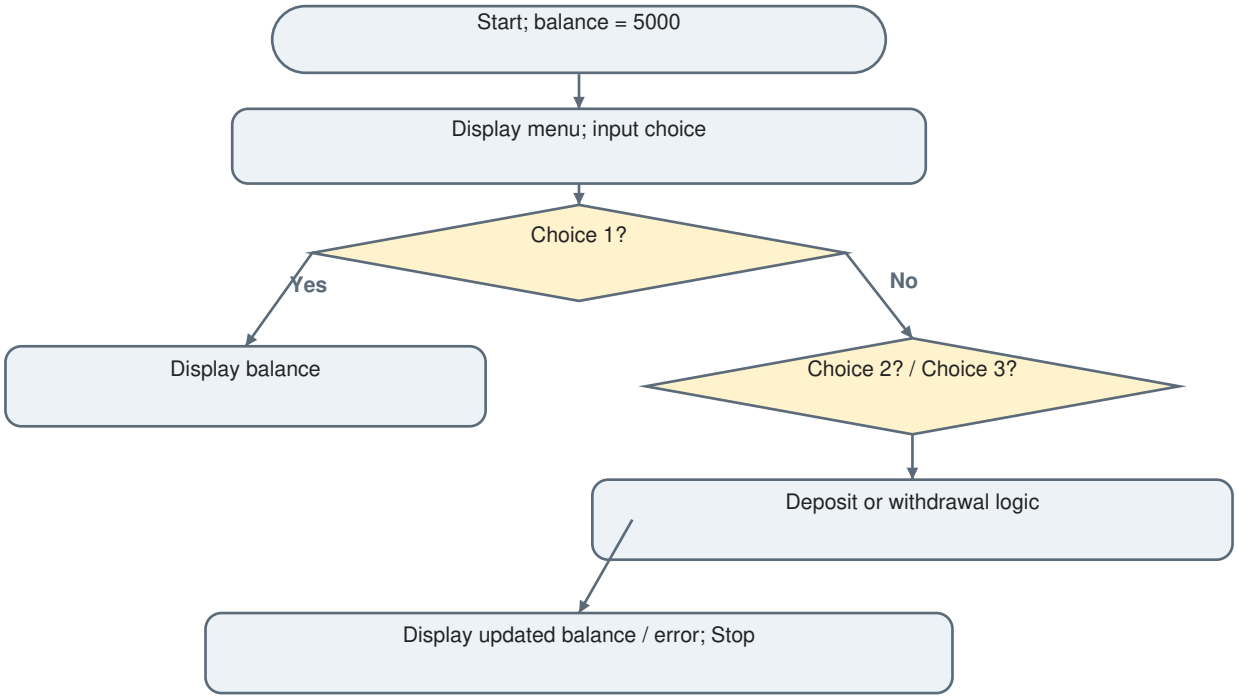
Conclusion

This case demonstrates the stated Python concept.

Experiment 7: Mini ATM System Simulation

FLOWCHART AND OUTPUT SCREENSHOT

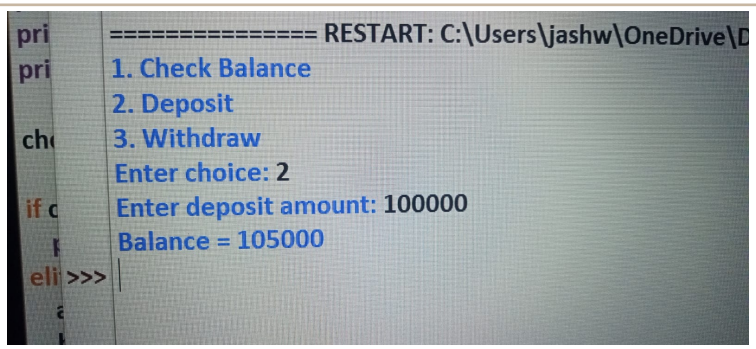
FLOWCHART



ASSUMPTIONS

The starting balance is 5,000. The user enters menu choice 1, 2, or 3 and a non-negative amount when needed.

PYTHON IDLE OUTPUT SCREENSHOT



Student Marks Total and Average

Python file: a8.py **Concept:** for loop

Problem Statement

Read marks for three subjects and calculate the total and average.

Objective

Use a for loop to repeat input three times and calculate the average.

Algorithm

1. Set total = 0.

2. Repeat three times: input marks and add them to total.

3. Calculate average = total / 3.

4. Display total and average.

Pseudocode

```
START
total = 0
FOR i = 1 TO 3
    INPUT marks
    total = total + marks
END FOR
average = total / 3
DISPLAY total, average
STOP
```

Python Source Code

```
total = 0

for i in range(1, 4):
    marks = int(input("Enter marks: "))
    total = total + marks

average = total / 3

print("Total marks =", total)
print("Average marks =", average)
```

Sample Input

```
Enter marks: 80
Enter marks: 90
Enter marks: 70
```

Expected Output — Text Only

```
Total marks = 240
Average marks = 80.0
```

Test Cases

Input / Test	Expected Result
80, 90, 70	Total marks = 240; Average marks = 80.0
60, 70, 80	Total marks = 210; Average marks = 70.0

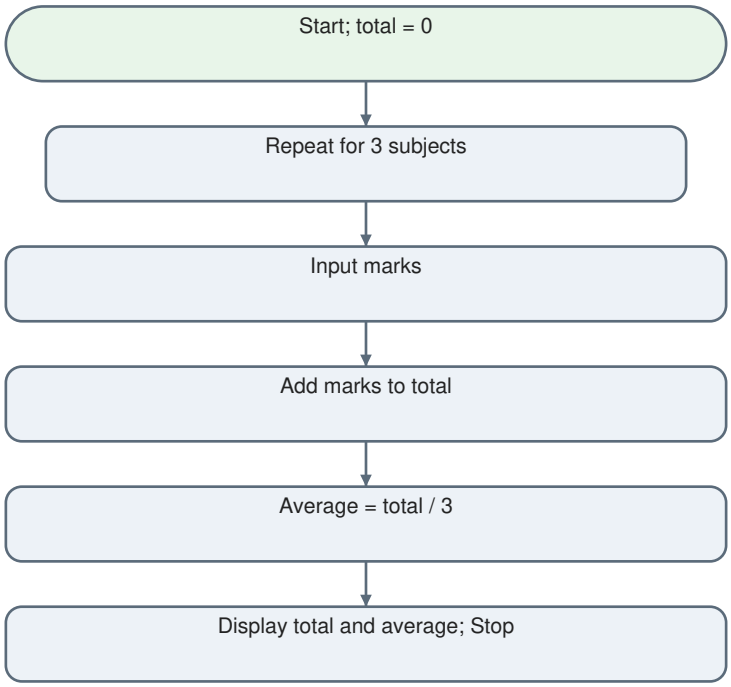
Conclusion

This case demonstrates the stated Python concept.

Experiment 8: Student Marks Total and Average

FLOWCHART AND OUTPUT SCREENSHOT

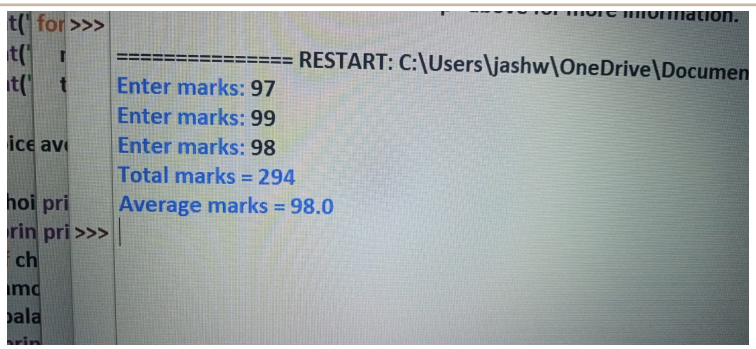
FLOWCHART



ASSUMPTIONS

The user enters marks for exactly three subjects. Marks are numeric values.

PYTHON IDLE OUTPUT SCREENSHOT



Weekly Power Consumption Tracker

Python file: a9.py **Concept:** while loop

Problem Statement

Add the units used over seven days.

Objective

Use a while loop and counter to collect seven daily unit values and calculate the total.

Algorithm

1. Set day = 1 and total = 0.

2. While day <= 7, input units and add them to total.

3. Increase day by 1 each loop.

4. Display total weekly units.

Pseudocode

```
START
day = 1
total = 0
WHILE day <= 7
    INPUT units
    total = total + units
    day = day + 1
END WHILE
DISPLAY total
STOP
```

Python Source Code

```
day = 1
total = 0

while day <= 7:
    units = int(input("Enter units used: "))
    total = total + units
    day = day + 1

print("Total weekly units =", total)
```

Sample Input

```
Enter units used: 10
Enter units used: 12
Enter units used: 8
Enter units used: 15
Enter units used: 10
Enter units used: 11
Enter units used: 14
```

Expected Output — Text Only

```
Total weekly units = 80
```

Test Cases

Input / Test	Expected Result
10, 12, 8, 15, 10, 11, 14	Total weekly units = 80

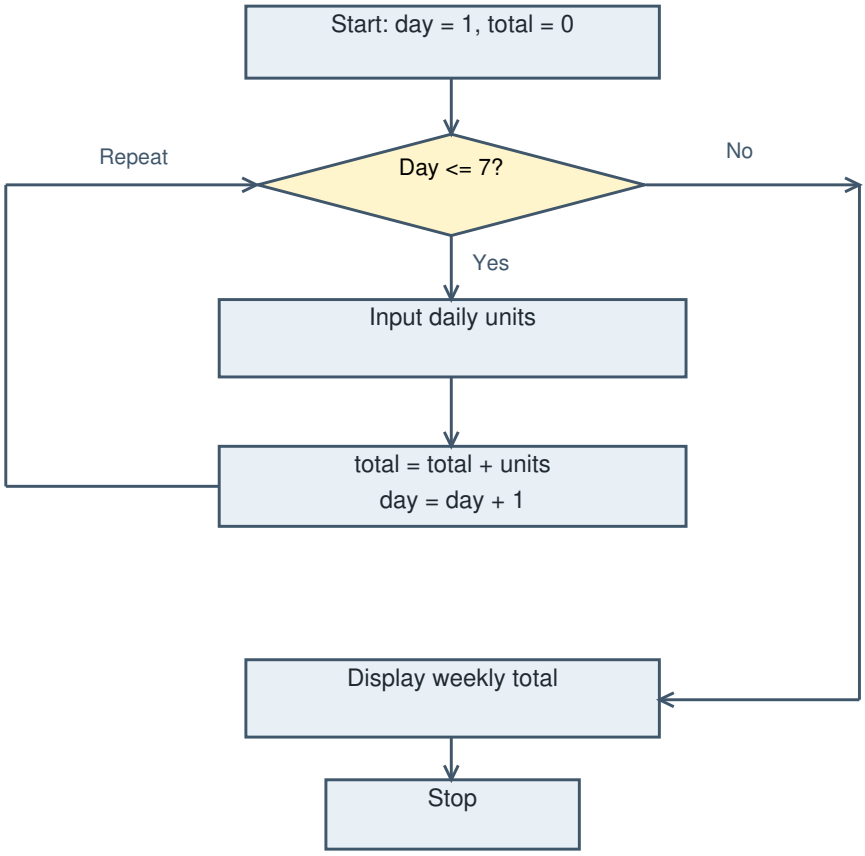
Conclusion

This case demonstrates the stated Python concept.

Experiment 9: Weekly Power Consumption Tracker

FLOWCHART AND OUTPUT SCREENSHOT

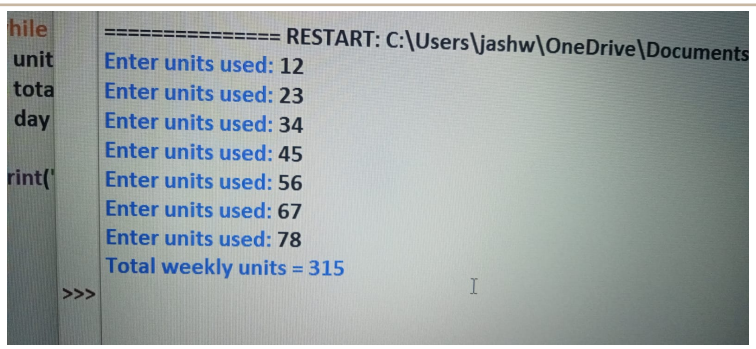
FLOWCHART



ASSUMPTIONS

The user enters one non-negative unit value for each of seven days.

PYTHON IDLE OUTPUT SCREENSHOT



Interactive Number Guessing Game

Python file: a10.py **Concept:** while loop

Problem Statement

Ask the user to guess a secret number until the correct number is entered.

Objective

Repeat the guessing prompt until the guess equals 7.

Algorithm

1. Set secret = 7.

2. Input a guess.

3. While guess != 7, print Wrong guess! and ask again.

4. When the guess is 7, print Correct guess!.

Pseudocode

```
START
secret = 7
INPUT guess
WHILE guess != secret
    DISPLAY "Wrong guess!"
    INPUT guess
END WHILE
DISPLAY "Correct guess!"
STOP
```

Python Source Code

```
secret = 7

guess = int(input("Guess a number from 1 to 10: "))

while guess != secret:
    print("Wrong guess!")
    guess = int(input("Try again: "))

print("Correct guess!")
```

Sample Input

```
Guess a number from 1 to 10: 4
Try again: 6
Try again: 7
```

Expected Output — Text Only

```
Wrong guess!
Wrong guess!
Correct guess!
```

Test Cases

Input / Test	Expected Result
4, 6, 7	Two Wrong guess! messages, then Correct guess!
7	Correct guess!

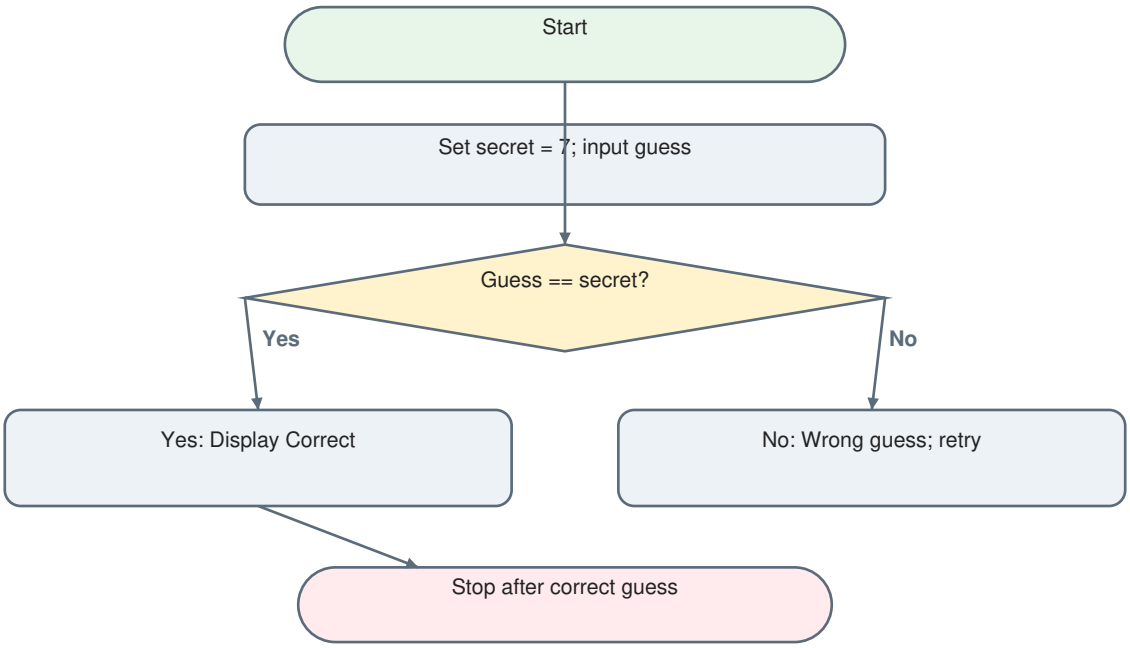
Conclusion

This case demonstrates the stated Python concept.

Experiment 10: Interactive Number Guessing Game

FLOWCHART AND OUTPUT SCREENSHOT

FLOWCHART



ASSUMPTIONS

The secret number is fixed at 7. The user enters integer guesses from 1 to 10 until correct.

PYTHON IDLE OUTPUT SCREENSHOT

